End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering - Mini-Project | Harsh Mohta (IMT2023106) | 30 October 2026

Repository: https://github.com/Harsh-Mohta9163/SPE_Mini_Project | Image: https://hub.docker.com/r/harsh9163/calculator-api

1. Overview

A Spring Boot 3.3.4 (Java 17, Maven) REST service performs add, subtract, multiply and divide and returns JSON. Every push to **main** on GitHub triggers a declarative Jenkins pipeline through a webhook. The pipeline builds and tests the code, builds a multi-stage Docker image tagged with the Jenkins build number, pushes it to Docker Hub and deploys it with Docker Compose behind an Nginx reverse proxy, then verifies the deployment with a smoke test against the health endpoint. An email reports success or failure. A code change reaches a running, health-checked deployment with no manual steps (build #7 took 1 min 24 s end to end).

2. Pipeline architecture

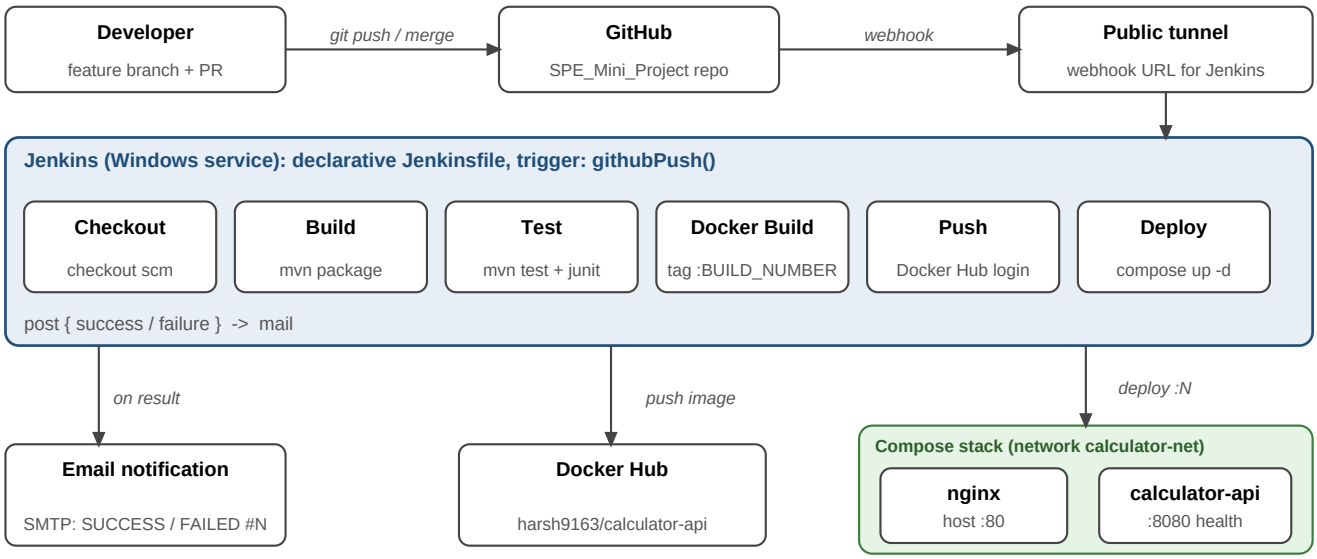


Figure 1: Flow from a developer push to a health-checked deployment. Shaded green: containers started by the Deploy stage.

3. Application

The code is split into layers: **CalculatorController** (HTTP only, constructor-injected service), **CalculatorService** (the arithmetic, no web code), the DTOs **CalculatorResponse** and **ErrorResponse** for the JSON shapes, and a **GlobalExceptionHandler** (*@RestControllerAdvice*) that maps a custom **DivisionByZeroException**, missing and mistyped parameters to HTTP 400. Spring Boot Actuator exposes **/actuator/health**, which Docker Compose and the Deploy stage use.

Request	Response
GET /api/v1/add?a=10&b=20	200 {"operation":"add","a":10.0,"b":20.0,"result":30.0} (subtract, multiply, divide alike)
GET /api/v1/divide?a=10&b=0	400 {"error":"Division by zero is not allowed","a":10.0,"b":0.0}
GET /api/v1/add?a=5	400 {"error":"Missing required parameter: b", ...}
GET /api/v1/add?a=abc&b=5	400 {"error":"Invalid value for parameter 'a': must be a number", ...}
GET /actuator/health	200 {"status":"UP", ...}

Tests (22, all passing): 13 JUnit 5 unit tests for the service, covering every operation plus divide by zero; and 9 Spring MVC integration tests (*MockMvc*) that send real requests through the controller and exception handler and check status codes and JSON fields. Surefire writes JUnit XML to *target/surefire-reports*, which Jenkins publishes.

4. Pipeline stages (Jenkinsfile)

Stage	What it does	Key detail
Checkout	Fetches the commit that triggered the build.	*checkout scm*; the job uses Pipeline script from SCM, so the pipeline is versioned with the code.
Build	Compiles and packages the executable JAR.	*mvn clean package -DskipTests -B*; clean gives a fresh build in the reused workspace.

Stage	What it does	Key detail
Test	Runs all tests, publishes results.	*mvn test -B*; a failing test makes Maven exit non-zero, which fails the stage. *post { always { junit } }* publishes the XML.
Docker Build	Builds the image from the multi-stage Dockerfile.	Tagged *harsh9163/calculator-api:${BUILD_NUMBER}* and *:latest*.
Push	Publishes both tags to Docker Hub.	*withCredentials* reads the access token from the Jenkins Credentials store; *--password-stdin* keeps it out of the logs.
Deploy	Starts the new version and verifies it.	*docker-compose up -d --force-recreate*, then *scripts/smoke_test.py* polls /actuator/health and checks every endpoint through Nginx.
post	Notifies the developer.	*mail* sends SUCCESS or FAILED with the build number, image tag and console link; *docker logout* always runs.

Trigger: *githubPush()*. GitHub posts to */github-webhook/* on a publicly reachable URL, since GitHub cannot reach localhost. Builds show "Started by GitHub push by Harsh-Mohta9163". **Idempotency:** *docker-compose up -d --force-recreate* replaces the containers of the same Compose project each run, so re-running the pipeline never leaves a half-deployed or duplicate stack.

5. Containerization and deployment

Dockerfile (multi-stage): stage 1 (*maven:3.9.6-eclipse-temurin-17-alpine*) copies only *pom.xml*, runs *dependency:go-offline*, then copies *src* and packages. Dependencies form their own cached layer, so a code change rebuilds in seconds. Stage 2 (*eclipse-temurin:17-jre-alpine*) creates a non-root system user *appuser*, copies only the JAR, declares *EXPOSE 8080* and runs as *USER appuser*, with a *HEALTHCHECK* on */actuator/health*. The final image is about 109 MB compressed on Docker Hub and contains no JDK, Maven or source code. *.dockerignore* excludes target/, .git/, docs and IDE files.

docker-compose.yml: two services on a user-defined bridge network *calculator-net*. *calculator-api* has no published port, only *expose: 8080*, plus a health check, *restart: unless-stopped* and environment variables (*SPRING_PROFILES_ACTIVE=prod*). *nginx:alpine* is the only public entry point (host 80 to container 80) and reverse-proxies to *calculator-api:8080* using Docker's service-name DNS. It starts only once the API is healthy (*depends_on: condition: service_healthy*). One *docker compose up* starts the whole stack.

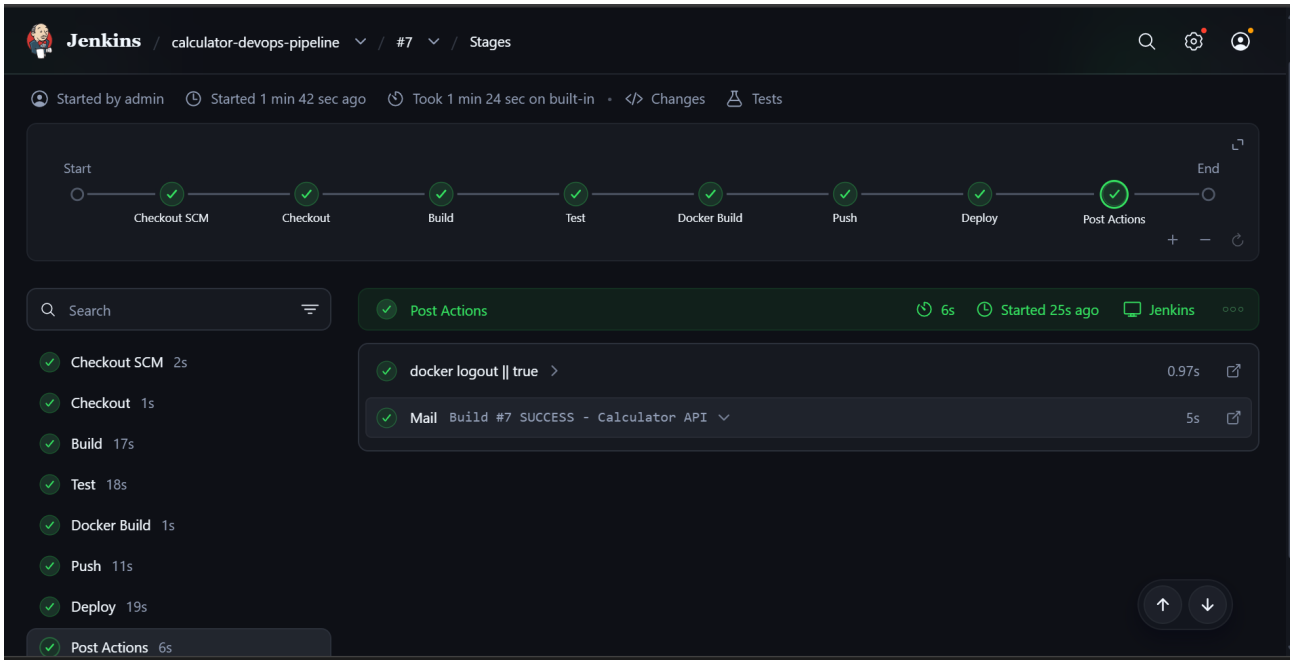
6. Version control

Work was done on feature branches (*feature/spring-boot-api*, *feature/docker-and-compose*, *feature/jenkins-and-pipeline-deliverables*) and merged into *main* through GitHub pull requests #1, #2 and #3, for a history of more than 30 small, descriptive commits. The Java/Maven .gitignore excludes target/ and IDE files. No secrets are in the repository: the Docker Hub access token lives only in the Jenkins Credentials store.

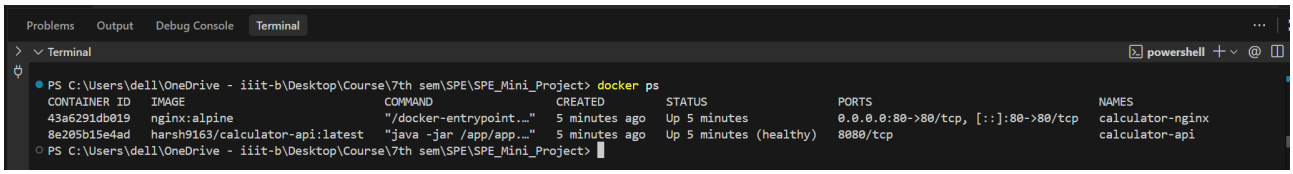
7. Challenges faced

Problem	Root cause	Resolution
Push failed: *unauthorized: incorrect username or password*	Docker Hub username is *harsh9163*, but the credential and image name used *harshmohta9163*; the stored username also had a leading space.	Fixed the image namespace, created a Read & Write access token and re-saved *dockerhub-creds*; tested with *docker login* by hand first.
Every build re-downloaded all Maven dependencies (~47 s)	*docker image prune -f* in the post step deleted the cached builder layers after each run.	Removed the prune; Docker Build dropped to about 1 s (see A1).
docker login -p warned that passing the token on the command line is insecure	The token was a command-line argument (and *echo* | *docker login* in cmd can add a trailing space).	Used *echo "$DH_PASS"* | *docker login --password-stdin* inside *withCredentials*.
Port conflict on 8080	Jenkins already listens on 8080, the same default port as the Spring Boot app.	Only Nginx publishes a host port (80); the API is exposed inside the network only.
Email notification failed with a connection error	Jenkins had no SMTP server configured (empty host, default port 25); Gmail also needs SSL and an app password, not the account password.	Set *smtp.gmail.com*, port 465 with SSL and a Gmail app password under Manage Jenkins, System, E-mail Notification; success and failure mails now arrive.

Appendix: Screenshots



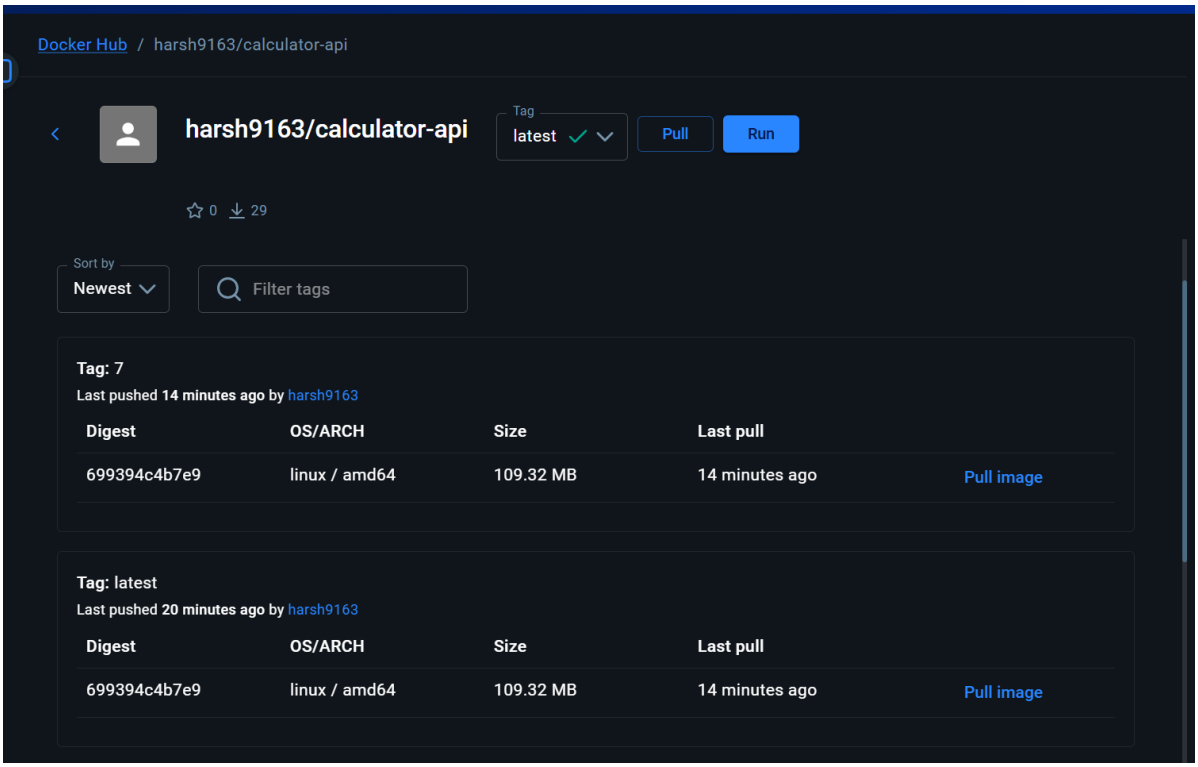
A1. Jenkins Stage View: build #7 green through all stages including Deploy (1 min 24 s).



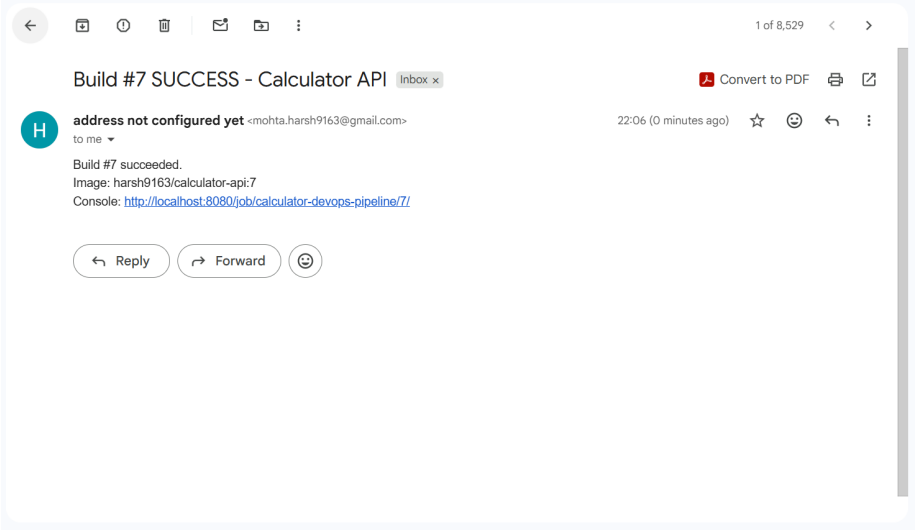
A2. docker ps after the pipeline: calculator-api (healthy) behind calculator-nginx on host port 80.

```
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" http://localhost/actuator/health
{"status":"UP","components":{"diskSpace":{"status":"UP","details":{"total":1081101176832,"free":1015900889088,"threshold":10485760,"path":"/app/.","exists":true}},"ping":{"status":"UP"}}}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/add?a=10&b=20"
{"operation":"add","a":10.0,"b":20.0,"result":30.0}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/subtract?a=20&b=5"
{"operation":"subtract","a":20.0,"b":5.0,"result":15.0}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/multiply?a=6&b=7"
{"operation":"multiply","a":6.0,"b":7.0,"result":42.0}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/divide?a=20&b=4"
{"operation":"divide","a":20.0,"b":4.0,"result":5.0}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/divide?a=10&b=0"
{"error":"Division by zero is not allowed","a":10.0,"b":0.0}
HTTP 400
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/add?a=abc&b=5"
{"error":"Invalid value for parameter 'a': must be a number","a":0.0,"b":0.0}
HTTP 400
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/add?a=5"
{"error":"Missing required parameter: b","a":0.0,"b":0.0}
HTTP 400
harsh@DESKTOP-D9MSMAP:~$ curl -s -w "\nHTTP %{http_code}\n" "http://localhost/api/v1/multiply?a=2.5&b=-4"
{"operation":"multiply","a":2.5,"b":-4.0,"result":-10.0}
HTTP 200
harsh@DESKTOP-D9MSMAP:~$
```

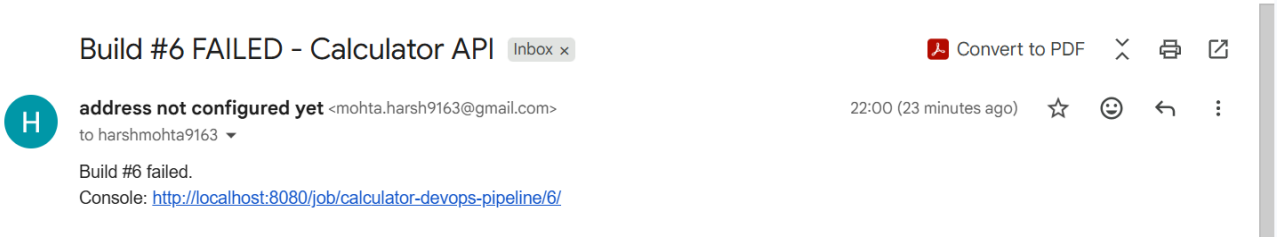
A3. API responses through Nginx: health, four operations (200) and three error cases (400).



A4. Docker Hub repository harsh9163/calculator-api with build-number (7) and latest tags.



A5. Email notification for a successful build (#7).



A6. Email notification for a failed build (#6, the Deploy stage failure described in section 7).